Lab Tasks

Task # 01:

An online shopping system calculates the final price of products. Different types of discounts are applied depending on the information available.

Requirements:
Create a class DiscountCalculator that uses compile time polymorphism (method overloading) to calculate the final price.

- If only price is provided => return the same price.
- If price and discount percentage are provided => apply percentage discount.
- If price, discount percentage, and coupon amount are provided => apply both percentage discount and coupon.

Implement overloaded methods calculatePrice() for these cases.

Task # 02:

A food delivery application calculates the total order price based on different parameters. If only the food price is provided, the system should return that price. If the food price and delivery distance are provided, the system should add delivery charges of 20 per kilometer. If the food price, distance, and coupon code are provided, then the system should first calculate the total price including delivery charges and then check the coupon: if the coupon is "SAVE10", a 10% discount should be applied to the total amount. Implement this system using method overloading in Java, ensuring that each overloaded method handles a different combination of parameters.

Task # 03:

A software company is developing a system to calculate electricity bills for different types of users. The system should use the same method name but perform different calculations depending on the information provided. If the user only provides the number of units consumed, the system should calculate the bill using a fixed rate of 10 per unit. If the user provides units and customer type (domestic or commercial), the system should apply different rates: domestic users are charged 10 per unit, while commercial users are charged 15 per unit. If the user also provides a loyalty status, then after calculating the bill, a 5% discount should be applied only if the customer is loyal. Design a Java class that uses compile-time polymorphism through method overloading to handle these different cases and return the final bill amount.

Task # 04:

A food delivery company calculates delivery cost differently based on delivery type. The base class Delivery contains a method calculateCost(double orderAmount). A StandardDelivery should charge a fixed delivery fee of 100, but if the order amount is greater than 2000, the delivery should be free. An ExpressDelivery should charge 200 as delivery fee, regardless of order amount. A ScheduledDelivery should charge 150, but if the order is scheduled more than one day earlier, the fee should be reduced to 100. Design a system using runtime polymorphism where each subclass overrides the delivery cost method, and the method that executes at runtime depends on the actual delivery object.

Task # 05:

A banking application calculates interest differently for different types of bank accounts. The base class BankAccount has a method calculateInterest(double balance) that returns the general interest amount. However, different account types calculate interest differently. A SavingsAccount should give 5% interest on the balance, while a CurrentAccount should give 2% interest only if the balance is greater than 50,000, otherwise it should return 0 interest. A FixedDepositAccount should give 8% interest, but if the balance is greater than 200,000, an additional 2% bonus interest should be

added. Design the class structure so that each account type overrides the interest calculation method and the correct method is executed at runtime depending on the object type.

Task # 06:

A smart home system controls different devices in a house. Each device behaves differently depending on the action requested. The base class Device has a method performAction(String action). Specific devices override this method to handle their own actions.

Device Requirements:

1. **Light** – "ON" turns on, "OFF" turns off, "DIM" sets brightness to 50%.
2. **Thermostat** – "SET_TEMPERATURE" sets a given temperature, "OFF" turns off, "BOOST" increases temperature by 5°C.
3. **SecurityCamera** – "RECORD" starts recording, "STOP" stops recording, "ALERT" triggers alarm if motion is detected.
4. **SmartSpeaker** – "PLAY" plays music, "STOP" stops music, "VOLUME_UP" or "VOLUME_DOWN" adjusts volume by 10 units.

Task:

- Design a base class Device with the method performAction(String action).
- Create subclasses: Light, Thermostat, SecurityCamera, SmartSpeaker, each overriding performAction() to implement device-specific behavior.
- Use runtime polymorphism: store different device objects in a parent class reference array/list and iterate to perform actions dynamically.
- Include logical handling of invalid actions (e.g., "DIM" on thermostat should be ignored).
- Example scenario: Turn all lights off, set thermostat to 22°C, start camera recording, and play music on the speaker.

Use method overriding to let each device respond appropriately at runtime, demonstrating dynamic behavior and logical handling of different actions.